A reproducible exact/approximate benchmark for 6F5 conservative-matrix-field matrix chains on Apple Silicon

David Vesterlund

21 August 2026

Summary

`apple6f5_benchmark` is a small, fully reproducible benchmark suite that measures how fast a long chain of integer 6×6 matrix products can be evaluated on an Apple-Silicon laptop, and how much precision is lost by the fast (floating-point) methods that make the computation tractable at scale. The matrix chain is not synthetic: every operator is a *primitive projective integer* matrix extracted from the corrected Ramanujan Dreams ${}_6F_5$ conservative-matrix-field (CMF) walk semantics (Elimelech et al. 2024; Weinbaum et al. 2025; Raayoni et al. 2021), so the benchmark exercises the exact kernel that underlies an active line of experimental-mathematics research. The suite ships three workloads (`sparse_x1`, `x6`, `dense_pos`) spanning 17- to 62-bit operators, and is archived at DOI 10.5281/zenodo.22045418 and `https://github.com/VesterlundCoder/applesilicon_rns_benchmark` (Vesterlund 2026).

The suite generates a deterministic, SHA-256-pinned corpus of operators together with an exact big-integer reference product, then times a broad set of back-ends against that reference on the same input:

- **Exact CPU:** Python `int`, SymPy (Meurer et al. 2017), python-flint / FLINT `fmpz_mat_mul` (The FLINT Team 2024), GMP `mpz_class` (Free Software Foundation 2024), Boost.Multiprecision `cpp_int` (Maddock and Kormanyos 2024), and a threaded residue-number-system (RNS) engine using 32-bit Montgomery multiplication (Montgomery 1985) with Chinese-Remainder reconstruction (Knuth 1997).
- **Exact GPU:** a Metal compute kernel that runs the same RNS chain, one `(prime, trajectory)` lane per thread.
- **Approximate:** Apple Accelerate `cblas_dgemm` (Apple Inc. 2024), NumPy (Harris et al. 2020), and a Metal *double-float* (df64) kernel (Dekker 1971; Hida et al. 2001) with power-of-two renormalisation after every operator, reported as projective error against the exact reference.

Every exact back-end must reproduce the reference SHA-256 or the run fails; every floating-point back-end reports its worst-case projective absolute and relative error. A separate driver benchmarks the repository's production Metal df64 CMF walk on *unique* deterministic trajectories, so that saturated-kernel throughput is never confused with real application throughput.

Statement of need

The Ramanujan Machine programme and its conservative-matrix-field generalisation search enormous spaces of matrix-valued recurrences for new closed forms of mathematical constants (Raayoni et al. 2021; Elimelech et al. 2024). The inner loop is always the same: multiply a long, structured chain of small rational/integer matrices and inspect the projective limit. Two facts make this loop a genuine engineering problem rather than a triviality:

1. The accumulated product grows to thousands of bits after only a few hundred steps (9546 bits after 1000 unit-axis ${}_6F_5$ operators in our default workload), so naive exact arithmetic is expensive and floating point silently loses the very digits the search depends on.

2. Modern experimental-mathematics pipelines run these walks *by the million* on consumer Apple-Silicon hardware, where the fastest path (Accelerate BLAS, or a Metal df64 kernel) is only *approximately* correct, and the exact path (GMP, FLINT, or an RNS decomposition) trades single-chain latency for GPU throughput.

Despite this, there is no shared, independently checkable benchmark that answers the practical questions a researcher actually asks: *How much faster is the approximate path? How much precision does it cost? When does the GPU win? Is a residue-number-system decomposition competitive with FLINT for this matrix size?* `apple6f5_benchmark` provides exactly that — a single `run_all.py` that builds the native/Metal binaries, generates the pinned corpus, runs all back-ends with warm-ups and repeat medians, verifies correctness, and writes machine-readable JSON plus exact dump files that a third party can re-check without trusting our timings.

The suite is deliberately self-contained and hardware-honest: it records the full system profile (chip, core split, compiler, BLAS provenance, git commit), distinguishes GPU-native from host-side timing, and separates three throughput measures that are easy to conflate: RNS **modular-kernel** throughput (residues only), **end-to-end exact** throughput (including parallel Chinese-Remainder reconstruction), and **unique-trajectory** application throughput. A CPU trajectory-parallel mode (GMP/FLINT/Accelerate across all cores) makes the CPU/GPU throughput comparison fair rather than pitting a saturated GPU against a single CPU thread. It is intended both as a reproducibility artefact for papers built on the Ramanujan Dreams engine and as a template others can adapt to benchmark exact-vs-approximate small-matrix chains on their own machines.

Functionality

Running

```
python run_all.py --repo-root <RamanujanDreams-RNS-Mac> --label M5-Max
```

produces, for the default `sparse_x1` workload (1000 operators, $z = 1/2$, rank 6):

- `corpus/` — the SHA-256-pinned operator binary, exact reference, and metadata;
- `python_results.jsonl`, `native_results.jsonl`, `metal_results.jsonl` — timing medians (plus MAD, min, max) for every back-end;
- `verification.jsonl` — per-dump correctness (exact) or projective error (floating), including the RNS modulus-capacity certificate;
- `production_df64.jsonl` — unique-trajectory application throughput;
- `system_metadata.json`, `run_manifest.json` — full provenance.

On an Apple M5 Max (18-core CPU: 6 super + 12 performance cores; 40-core GPU (Apple Inc. 2025)) the exact single-chain result is fastest with GMP ($\$ 8.5ms), $ *theapproximatesingle* $ − *chainresultisfastestwithAccelerate*(\$0.27 ms at 2.4×10^{-15} projective error), and the RNS-GPU modular kernel saturates at $ $3kresidue-trajectories/GPU-second.Crucially, onceper-trajectoryChinese-Remainderreconstructionisincludedtheexact RNS path($0.5k/s) fallsbelow trajectory-parallel CPU GMP(\$1.5k/s), so the GPU wins on exact arithmetic only when reconstruction is amortised. All numbers are regenerated end-to-end by the command above.

Acknowledgements

We thank the FLINT, GMP, Boost, and NumPy communities, whose libraries are the exact-arithmetic baselines in this work.

References

Apple Inc. 2024. *Accelerate Framework (BLAS/LAPACK, vecLib).*

Apple Inc. 2025. *MacBook Pro (14-Inch and 16-Inch, M5 Pro or M5 Max) — Technical Specifications*.

Dekker, T. J. 1971. "A Floating-Point Technique for Extending the Available Precision." *Numerische Mathematik* 18 (3): 224–42. https://doi.org/10.1007/BF01397083.

Elimelech, Rotem, Ofir David, Carlos De la Cruz Mengual, et al. 2024. "Algorithm-Assisted Discovery of an Intrinsic Order Among Mathematical Constants." *Proceedings of the National Academy of Sciences* 121 (25): e2321440121. https://doi.org/10.1073/pnas.2321440121.

Free Software Foundation. 2024. *The GNU Multiple Precision Arithmetic Library (GMP)*.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. "Array Programming with NumPy." *Nature* 585: 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Hida, Yozo, Xiaoye S. Li, and David H. Bailey. 2001. "Algorithms for Quad-Double Precision Floating Point Arithmetic." *Proceedings of the 15th IEEE Symposium on Computer Arithmetic (ARITH-15)*, 155–62. https://doi.org/10.1109/ARITH.2001.930115.

Knuth, Donald E. 1997. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3rd ed. Addison-Wesley.

Maddock, John, and Christopher Kormanyos. 2024. *Boost.multiprecision*.

Meurer, Aaron, Christopher P. Smith, Mateusz Paprocki, et al. 2017. "SymPy: Symbolic Computing in Python." *PeerJ Computer Science* 3: e103. https://doi.org/10.7717/peerj-cs.103.

Montgomery, Peter L. 1985. "Modular Multiplication Without Trial Division." *Mathematics of Computation* 44 (170): 519–21. https://doi.org/10.1090/S0025-5718-1985-0777282-X.

Raayoni, Gal, Shahar Gottlieb, Yahel Manor, et al. 2021. "Generating Conjectures on Fundamental Constants with the Ramanujan Machine." *Nature* 590 (7844): 67–73. https://doi.org/10.1038/s41586-021-03229-4.

The FLINT Team. 2024. *FLINT: Fast Library for Number Theory*.

Vesterlund, David. 2026. *Apple Silicon ${}_6F_5$ RNS Benchmark: Code, Data, and Results*. https://doi.org/10.5281/zenodo.22045418.

Weinbaum, Ofir, Elyasheev Leibtag, Rotem Kalisch, Michael Shalyt, and Ido Kaminer. 2025. "On Conservative Matrix Fields: Continuous Asymptotics and Arithmetic." *arXiv Preprint arXiv:2507.08138*, ahead of print. https://doi.org/10.48550/arXiv.2507.08138.